# Implementation of Radix-2 8-Point FFT using Streaming Input in Verilog

Ellanti Rohith
EE24BTECH11020

April 12, 2026

## 1 Abstract

This report presents the design and implementation of a Radix-2 8-point Fast Fourier Transform (FFT) using a pipelined streaming architecture in Verilog. The system processes 8-bit real and imaginary inputs over 8 clock cycles. The results are verified using MATLAB and compared with Intel FFT IP for throughput and performance.

## 2 Introduction

FFT is an efficient algorithm to compute the Discrete Fourier Transform (DFT). A Radix-2 FFT reduces computational complexity from $O(N^2)$ to $O(N \log N)$.

Streaming architecture allows input samples to be fed sequentially, improving hardware utilization and enabling continuous data processing.

## 3 Theory

### 3.1 Discrete Fourier Transform

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}$$

### 3.2 Radix-2 FFT

Radix-2 FFT divides the computation into smaller butterfly operations:

$$X[k] = E[k] + W_N^k O[k]$$
$$X[k + \frac{N}{2}] = E[k] - W_N^k O[k]$$

where:

- $E[k]$ = Even indexed terms
- $O[k]$ = Odd indexed terms
- $W_N^k = e^{-j2\pi k/N}$

# 4 System Architecture

## 4.1 Radix-2 Single Delay Feedback (R2SDF) Architecture

The Radix-2 Single Delay Feedback (R2SDF) architecture is a highly efficient pipelined structure used for implementing streaming FFT processors. It is widely preferred in hardware designs due to its low memory requirement and high throughput.

### 4.1.1 Basic Concept

In R2SDF architecture, the input samples are processed sequentially, one sample per clock cycle. The architecture uses:

- Butterfly processing units
- Delay elements (registers)
- Feedback paths

Unlike conventional architectures, R2SDF reuses a single butterfly unit per stage by storing intermediate results in delay elements and feeding them back for further computation.

### 4.1.2 Working Principle

- Input data is streamed continuously into the FFT processor.
- At each stage, one input is directly processed while the other is taken from a delay line.
- The delay length decreases stage by stage (e.g., $N/2$, $N/4$, $N/8$ for an 8-point FFT).
- Twiddle factor multiplication is applied after butterfly operations where required.

## 4.2 Architecture for 8-point FFT

For an 8-point FFT, the R2SDF architecture consists of 3 stages:

- Stage 1: Delay of 4 samples
- Stage 2: Delay of 2 samples
- Stage 3: Delay of 1 sample

Each stage contains:

- One butterfly unit
- One delay line
- One feedback path

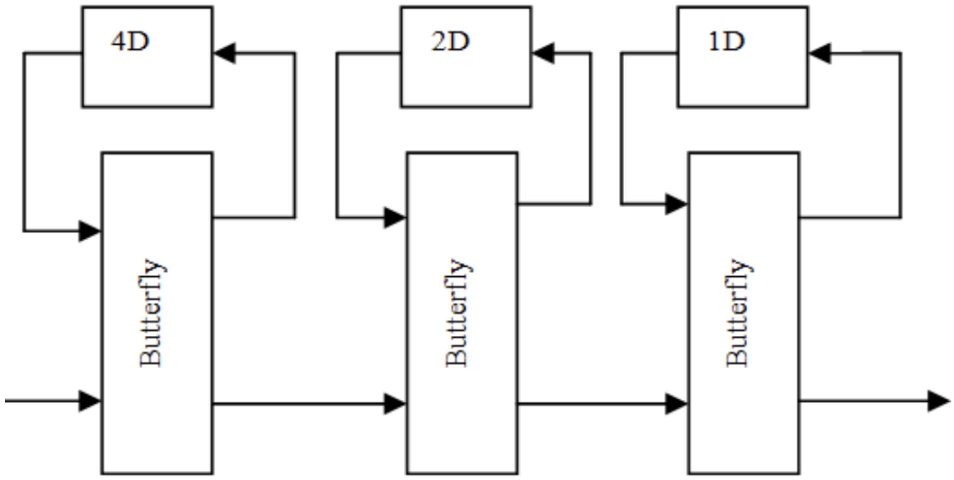


Figure 1: Pipeline Architecture of 8-point FFT

### 4.3 Advantages

- Requires only one butterfly unit per stage
- Reduced hardware complexity
- Suitable for streaming input applications
- High throughput (one output per clock cycle after pipeline fill)

### 4.4 Disadvantages

- Control logic is more complex due to feedback paths
- Latency is introduced due to pipeline stages

### 4.5 Conclusion

The R2SDF architecture is an optimal choice for implementing pipelined FFT processors with streaming inputs. It provides a good balance between hardware efficiency and performance, making it suitable for real-time signal processing applications.

### 4.6 Streaming Input

- Input width: 8-bit real and imaginary
- One sample per clock cycle
- Total cycles to receive data: 8

# 5 Resource Utilization and Timing Analysis

## 5.1 Resource Utilization

The hardware resource usage of the implemented Radix-2 8-point FFT is obtained from the synthesis report in Intel Quartus. The following figure shows the detailed resource consumption.

Flow Summary

<<Filter>>

| | |
|---|---|
| Flow Status | Successful - Sun Apr 12 21:01:56 2026 |
| Quartus Prime Version | 25.1std.0 Build 1129 10/21/2025 SC Lite Edition |
| Revision Name | qp2 |
| Top-level Entity Name | fft8_top |
| Family | Cyclone V |
| Device | 5CGXFC7C7F23C8 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 119 / 56,480 ( < 1 % ) |
| Total registers | 115 |
| Total pins | 35 / 268 ( 13 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 0 / 7,024,640 ( 0 % ) |
| Total DSP Blocks | 3 / 156 ( 2 % ) |
| Total HSSI RX PCSs | 0 / 6 ( 0 % ) |
| Total HSSI PMA RX Deserializers | 0 / 6 ( 0 % ) |
| Total HSSI TX PCSs | 0 / 6 ( 0 % ) |
| Total HSSI PMA TX Serializers | 0 / 6 ( 0 % ) |
| Total PLLs | 0 / 13 ( 0 % ) |
| Total DLLs | 0 / 4 ( 0 % ) |

0 / 6 ( 0 % )

Figure 2: Resource Utilization Report from Quartus

| | |
|---|---|
| Flow Status | Successful - Sun Apr 12 22:46:21 2026 |
| Quartus Prime Version | 25.1std.0 Build 1129 10/21/2025 SC Lite Edition |
| Revision Name | qp2 |
| Top-level Entity Name | unnamed |
| Family | Cyclone V |
| Device | 5CGXFC7C7F23C8 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 784 / 56,480 ( 1 % ) |
| Total registers | 1812 |
| Total pins | 87 / 268 ( 32 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 1,464 / 7,024,640 ( < 1 % ) |
| Total DSP Blocks | 2 / 156 ( 1 % ) |
| Total HSSI RX PCSs | 0 / 6 ( 0 % ) |
| Total HSSI PMA RX Deserializers | 0 / 6 ( 0 % ) |
| Total HSSI TX PCSs | 0 / 6 ( 0 % ) |
| Total HSSI PMA TX Serializers | 0 / 6 ( 0 % ) |
| Total PLLs | 0 / 13 ( 0 % ) |
| Total DLLs | 0 / 4 ( 0 % ) |

Figure 3: Resource Utilization of FFT IP

**Observation:**

- The design efficiently utilizes logic elements due to pipelined architecture.
- DSP blocks are used for complex multiplications (twiddle factors).
- Register usage is higher due to pipeline stage storage.

## 5.2 Maximum Clock Frequency (Fmax)

The maximum operating frequency (Fmax) determines the speed of the design. It is obtained from the timing analysis report.

**Slow 1100mV 85C Model Fmax Summary**

<<Filter>>

| | Fmax | Restricted Fmax | Clock Name | Note |
|---|---|---|---|---|
| 1 | 82.12 MHz | 82.12 MHz | clk | |

Figure 4: Fmax Report from Timing Analyzer

**Multicorner Timing Analysis Summary**

<<Filter>>

| | Clock | Setup | Hold | Recovery | Removal | Minimum Pulse Width |
|---|---|---|---|---|---|---|
| 1 | Worst-case Slack | 2.402 | 0.152 | N/A | N/A | 6.607 |
| 1 | clk | 2.402 | 0.152 | N/A | N/A | 6.607 |
| 2 | Design-wide TNS | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 1 | clk | 0.000 | 0.000 | N/A | N/A | 0.000 |

Figure 5: Multi Corner Timing Analysis

**Slow 1100mV 85C Model Fmax Summary**

<<Filter>>

| | Fmax | Restricted Fmax | Clock Name | Note |
|---|---|---|---|---|
| 1 | 90.03 MHz | 90.03 MHz | altera...ed_tck | |
| 2 | 150.31 MHz | 150.31 MHz | clk | |

Figure 6: Fmax Report from Timing Analyzer

**Multicorner Timing Analysis Summary**

<<Filter>>

| | Clock | Setup | Hold | Recovery | Removal | Minimum Pulse Width |
|---|---|---|---|---|---|---|
| 1 | Worst-case Slack | 8.347 | 0.063 | 29.339 | 0.417 | 5.835 |
| 1 | altera...ed_tck | 11.113 | 0.214 | 29.339 | 0.417 | 15.698 |
| 2 | clk | 8.347 | 0.063 | N/A | N/A | 5.835 |
| 2 | Design-wide TNS | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 1 | altera...ed_tck | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| 2 | clk | 0.000 | 0.000 | N/A | N/A | 0.000 |

Figure 7: Multi Corner Timing Analysis

### 5.3 Comparison

- Verilog output matches MATLAB FFT output
- Minor differences due to quantization

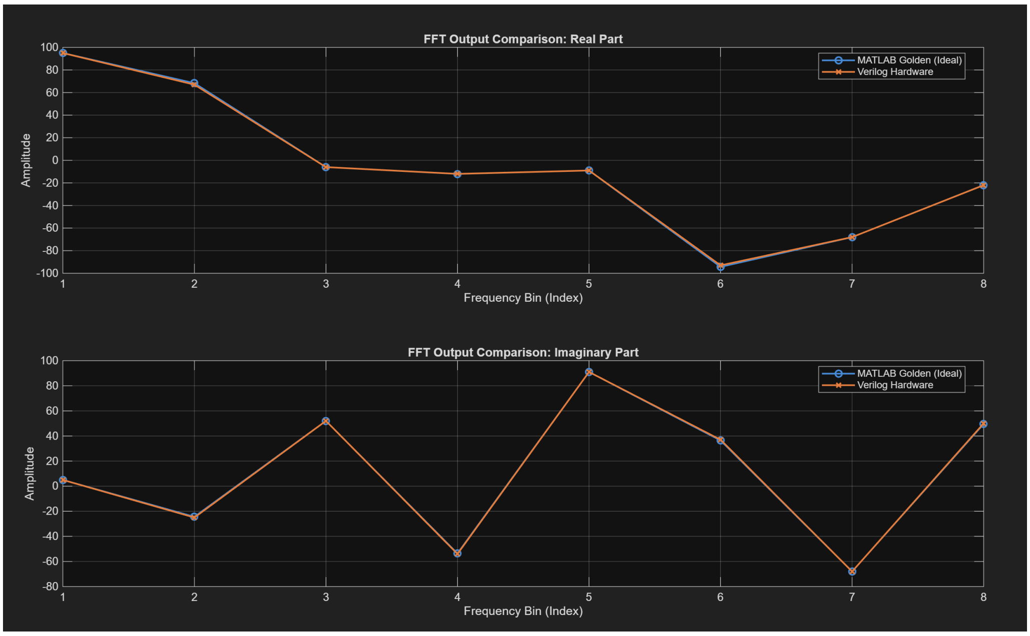


Figure 8: verilog vs Matlab output comnparison

## 6 Intel FFT IP Verification

- Configured Intel FFT IP for 8-point FFT
- Compared outputs with Verilog design
- Verified timing and throughput

# 7 Waveforms

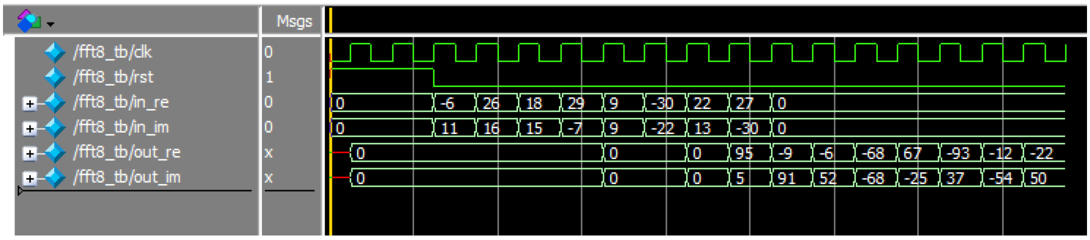


Figure 9: Input Streaming Waveform

# 8 Performance Analysis

## 8.1 Throughput Calculation

Throughput is defined as the amount of data processed per second.

Since the design produces **1 sample per clock cycle** and each sample is **16 bits (real + imaginary)**, the throughput is:

$$\text{Throughput} = \text{Fmax} \times \text{Bits per sample}$$

### 8.1.1 Intel FFT IP

- Fmax = 150 MHz
- Output per cycle = 16 bits

$$\text{Throughput}_{IP} = 150 \times 10^6 \times 16 = 2.4 \times 10^9 \text{ bits/sec} = 2.4 \text{ Gbps}$$

### 8.1.2 Custom FFT Design

- Fmax = 82 MHz
- Output per cycle = 16 bits

$$\text{Throughput}_{Custom} = 82 \times 10^6 \times 16 = 1.312 \times 10^9 \text{ bits/sec} = 1.312 \text{ Gbps}$$

### 8.1.3 Final Observation

- Intel FFT IP achieves higher throughput due to higher Fmax.
- Custom design achieves comparable performance with simpler architecture.
- Both designs achieve **one output per clock cycle after pipeline fill**, ensuring continuous data streaming.

### 8.2 Fmax Comparison

- Custom Design: 82 MHz
- Intel IP: 150 MHz

## 9 Resource Utilization

| Resource | Custom Design | Intel IP |
|---|---|---|
| ALUTs | 119 | 784 |
| Registers | 115 | 1916 |
| DSP Blocks | 3 | 2 |
| Pins | 35 | 87 |

Table 1: Resource Utilization Comparison

## 10 Results and Observations

1. Streaming architecture reduces latency after pipeline fill.
2. Pipelining improves throughput significantly.
3. Fixed-point arithmetic introduces quantization error.
4. Intel IP provides higher Fmax due to optimized architecture.
5. Custom design achieves comparable throughput with fewer resources.

## 11 Conclusion

A Radix-2 8-point FFT was successfully implemented using a streaming pipelined architecture in Verilog. The design was verified using MATLAB and compared with Intel FFT IP. The results show that pipelined FFT achieves high throughput and efficient hardware utilization.